

007 — Troubleshooting FAQ

 This document covers the most common issues you'll encounter during the bootcamp. Find your error message below and follow the fix step by step.

Tip: Use `Ctrl+F` (or `Cmd+F` on Mac) to search for your specific error message.

Table of Contents

1. [SSH Issues](#)
 2. [Docker Daemon Issues](#)
 3. [Port Conflicts](#)
 4. [Container Issues](#)
 5. [SSL / HTTPS Issues](#)
 6. [Database Issues](#)
 7. [GitHub / Git Issues](#)
 8. [GitHub Actions / CI-CD Issues](#)
 9. [DNS Issues](#)
 10. [WSL-Specific Issues](#)
 11. [Resource / Performance Issues](#)
 12. [Nuclear Options \(Start Over\)](#)
-

1. SSH Issues

1.1 "Permission denied (publickey)"

Full error:

```
Permission denied (publickey).
fatal: Could not read from remote repository.
```

Cause: SSH key is not being sent to the server, or the server doesn't recognize it.

Fix — Step by step:

```
# Step 1: Is your SSH agent running?
echo $SSH_AUTH_SOCK
# If empty → agent not running

# Step 2: Start agent and add key
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Step 3: Verify key is loaded
```

```
ssh-add -l
# Should show your key fingerprint

# Step 4: Test connection
ssh -T git@github.com
# or
ssh -vT git@github.com # verbose mode for debugging
```

If it still fails:

```
# Check the key exists
ls -la ~/.ssh/id_ed25519
ls -la ~/.ssh/id_ed25519.pub

# Check permissions (MUST be exact)
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519
chmod 644 ~/.ssh/id_ed25519.pub

# Verify your public key matches what's on GitHub
cat ~/.ssh/id_ed25519.pub
# Compare with: https://github.com/settings/keys
```

Still not working? Regenerate:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
# Then add the new public key to GitHub again
```

1.2 "Connection refused" (SSH to VPS)

Full error:

```
ssh: connect to host 167.71.123.45 port 22: Connection refused
```

Causes & Fixes:

Cause	Fix
Droplet isn't fully booted	Wait 1-2 minutes after creation
SSH service not running	Access via DigitalOcean Console (web browser)
Wrong IP address	Verify IP on DO dashboard
Port 22 blocked by firewall	Use DO Console to fix UFW

Access via DigitalOcean Console (Emergency Access):

1. Go to <https://cloud.digitalocean.com/droplets>
2. Click on your Droplet
3. Click **Console** (top-right, "Access" tab)
4. This opens a web-based terminal — no SSH needed!

Fix firewall from console:

```
ufw allow 22/tcp
ufw reload
```

1.3 "Connection timed out" (SSH to VPS)

Full error:

```
ssh: connect to host 167.71.123.45 port 22: Operation timed out
```

Causes & Fixes:

```
# 1. Verify the IP is correct
ping YOUR_DROPLET_IP
# If no response → wrong IP or server is down

# 2. Check if port 22 is reachable
nc -zv YOUR_DROPLET_IP 22
# If timeout → firewall issue or server down

# 3. Try from a different network (phone hotspot)
# Some corporate/school networks block port 22

# 4. If your network blocks port 22, use SSH over HTTPS port:
# Add to ~/.ssh/config:
Host vps
  HostName YOUR_DROPLET_IP
  User root
  Port 22
  IdentityFile ~/.ssh/id_ed25519
  # If port 22 blocked, try D0 Console
```

If server is down: Reboot via DigitalOcean dashboard → Droplet → Power → Power Cycle.

1.4 "WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED"

Full error:

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Host key for 167.71.123.45 has changed...
```

Cause: You destroyed and recreated the Droplet with the same IP, or the server was reimaged.

Fix:

```
# Remove the old host fingerprint
ssh-keygen -R YOUR_DROPLET_IP

# Try connecting again (will ask to accept new fingerprint)
ssh root@YOUR_DROPLET_IP
# Type "yes" when prompted
```

1.5 "Too many authentication failures"

Full error:

```
Received disconnect from 167.71.123.45 port 22:2: Too many authentication
failures
```

Cause: SSH agent is offering multiple keys, and the server rejects all before reaching the right one.

Fix:

```
# Specify exact key
ssh -i ~/.ssh/id_ed25519 root@YOUR_DROPLET_IP

# Or clear agent and add only the correct key
ssh-add -D
ssh-add ~/.ssh/id_ed25519

# Or set in SSH config:
Host vps
  HostName YOUR_DROPLET_IP
  User root
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes      # Only try this key
```

2. Docker Daemon Issues

2.1 "Cannot connect to the Docker daemon"

Full error:

```
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the
docker daemon running?
```

Fix for WSL (Windows):

```
# Start Docker service
sudo service docker start

# Verify
sudo service docker status
# Should show: * Docker is running

# If it fails to start:
sudo dockerd &
# Check error messages
```

Fix for macOS:

- Open Docker Desktop application
- Wait for whale icon in menu bar to stop animating
- If stuck: Quit Docker Desktop → Reopen

Fix for VPS (Linux):

```
# Start Docker
systemctl start docker

# Check status
systemctl status docker

# If failed, check logs:
journalctl -u docker --since "5 min ago"

# Reinstall if corrupted:
apt remove -y docker-ce docker-ce-cli
apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
systemctl start docker
```

2.2 "Got permission denied while trying to connect to the Docker daemon socket"

Full error:

```
Got permission denied while trying to connect to the Docker daemon socket
at unix:///var/run/docker.sock:
permission denied
```

Cause: Your user isn't in the `docker` group.

Fix:

```
# Add yourself to docker group
sudo usermod -aG docker $USER

# CRITICAL: You must log out and log back in!
# On WSL: close terminal completely and reopen
# On VPS: exit SSH and reconnect
exit
```

After reconnecting:

```
# Verify
groups
# Should include "docker"

# Test
docker ps
```

Quick workaround (not recommended long-term):

```
sudo docker ps
sudo docker compose up -d
```

2.3 "docker compose" command not found

Full error:

```
docker: 'compose' is not a docker command.
```

Cause: Docker Compose plugin not installed (you may have the old `docker-compose` v1).

Fix:

```
# Install Docker Compose plugin
sudo apt install -y docker-compose-plugin

# Verify
docker compose version
# Should show: Docker Compose version v2.x.x
```

If you have old docker-compose (v1):

```
# Check version
docker-compose --version

# Old v1 uses hyphen: docker-compose up -d
# New v2 uses space: docker compose up -d

# Remove old version and use new:
sudo apt remove docker-compose
sudo apt install -y docker-compose-plugin
```

2.4 Docker Daemon Won't Start (WSL)

Full error:

```
Cannot connect to the Docker daemon. Is the docker daemon running?
failed to start daemon: pid file found, ensure docker is not running or
delete /var/run/docker.pid
```

Fix:

```
# Remove stale PID file
sudo rm -f /var/run/docker.pid

# Kill any orphaned processes
sudo killall dockerd 2>/dev/null
sudo killall containerd 2>/dev/null

# Start fresh
sudo service docker start

# Verify
docker ps
```

3. Port Conflicts

3.1 "Port is already allocated" / "Address already in use"

Full error:

```
Error starting userland proxy: listen tcp4 0.0.0.0:80: bind: address
already in use
```

or

```
Bind for 0.0.0.0:3000 failed: port is already allocated
```

Find what's using the port:

```
# On Linux/WSL:
sudo ss -tlnp | grep :80
# or
sudo lsof -i :80

# On macOS:
sudo lsof -i :80
```

Fix options:

```
# Option 1: Kill the process using the port
sudo kill $(sudo lsof -t -i :80)

# Option 2: Stop the conflicting container
docker stop $(docker ps -q --filter "publish=80")

# Option 3: Stop ALL containers
docker stop $(docker ps -q)

# Option 4: Use a different port
# In docker-compose.yml, change:
ports:
  - "8080:80"    # Use 8080 instead of 80
```

Common port conflicts:

Port	Usually used by
80	Apache, Nginx, another Caddy, or Docker container
443	Same as above (HTTPS)

Port	Usually used by
3000	Next.js dev server, another Node app
5010	Another .NET API instance
5432	PostgreSQL (local installation)
8080	Various dev servers, Adminer

3.2 "Port 5432 already in use" (PostgreSQL)

Cause: Local PostgreSQL installation is running.

```
# Stop local PostgreSQL
# Ubuntu/WSL:
sudo service postgresql stop

# macOS:
brew services stop postgresql

# Or just use a different port in docker-compose:
ports:
  - "5433:5432"    # Map to 5433 on host
```

4. Container Issues

4.1 Container Keeps Restarting (Restart Loop)

Symptoms:

```
docker compose ps
NAME                STATUS
kanban-api          Restarting (1) 5 seconds ago
```

Diagnosis:

```
# Check exit code
docker inspect kanban-api --format '{{.State.ExitCode}}'
# 0 = normal exit, 1 = app error, 137 = killed (OOM), 139 = segfault

# Check logs IMMEDIATELY (before it restarts and clears them)
docker logs kanban-api --tail 50

# Check events
docker events --since 5m --filter container=kanban-api
```

Common causes & fixes:

Exit Code	Cause	Fix
1	Application error (crash)	Read logs for stack trace
137	Out of memory (OOM killed)	Increase Droplet RAM or reduce services
139	Segfault	Rebuild image
0 + restart	App starts then exits normally	Check if CMD/ENTRYPOINT is correct

Most common fix for API container:

```
# Usually a database connection issue
docker compose logs api | grep -i "error\|fail\|exception"

# Common messages:
# "Connection refused" → DB not reachable (check trusted sources)
# "password authentication failed" → Wrong credentials in .env
# "database does not exist" → DB name wrong in connection string
# "SSL connection error" → Missing SSL Mode=Require in connection string
```

Fix for environment variable issues:

```
# Verify .env is being read
docker compose config | grep DATABASE

# Recreate containers with updated .env
docker compose down
docker compose up -d
```

4.2 Container Exits Immediately (Status: "Exited")

Diagnosis:

```
docker compose ps -a
# Shows: kanban-api    Exited (1) 30 seconds ago

docker compose logs api
```

Common causes:

1. Missing environment variables:

```
# Check what's actually set inside the container
docker compose run --rm api env | sort
```

2. Database migration failed:

```
# Look for migration errors
docker compose logs api | grep -i "migration\|migrate"
```

3. Wrong base image platform:

```
# If you built on Mac M1/M2 but running on VPS (amd64):
# Rebuild with correct platform:
docker buildx build --platform linux/amd64 -t myapp .
```

4.3 "Image not found" / "Pull access denied"

Full error:

```
Error response from daemon: Head
"https://ghcr.io/v2/proxeon/bootcamp/api/manifests/latest": unauthorized
```

or

```
Error response from daemon: pull access denied for
ghcr.io/proxeon/bootcamp/api
```

Cause: Not logged into GHCR, or token doesn't have `read:packages` permission.

Fix:

```
# Login to GHCR
echo "ghp_YOUR_TOKEN" | docker login ghcr.io -u YOUR_GITHUB_USERNAME --
password-stdin

# Verify login worked
cat ~/.docker/config.json | grep ghcr

# Try pulling again
docker compose pull
```

If login succeeds but pull still fails:

- Verify your PAT has `read:packages` scope
 - Verify the package is accessible to you (check GitHub org settings)
 - Verify the image path is correct: `ghcr.io/proxeon/bootcamp/api:latest`
-

4.4 "No space left on device"

Full error:

```
Error response from daemon: failed to mkdir: no space left on device
```

Fix:

```
# Check disk usage
df -h /

# Clean up Docker (biggest space saver)
docker system prune -af
docker builder prune -af
docker volume prune -f

# Check what's using space
du -sh /var/lib/docker/*
ncdu /

# Remove old logs
truncate -s 0 /var/lib/docker/containers/*/*-json.log

# After cleanup, try again
docker compose up -d
```

Prevent future issues:

```
# Add log rotation to /etc/docker/daemon.json
cat > /etc/docker/daemon.json << 'EOF'
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
EOF

systemctl restart docker
```

4.5 Container Can't Reach Another Container

Symptom: API can't connect to database, or Gateway can't reach API.

```
# Check both containers are on the same network
docker network inspect $(docker network ls -q --filter name=kanban)

# Test connectivity from one container to another
docker exec kanban-api sh -c "nc -zv postgres 5432"
docker exec kanban-gateway sh -c "nc -zv api 5010"

# Check DNS resolution
docker exec kanban-api sh -c "nslookup postgres"
```

Common causes:

- Containers in different docker-compose files (different networks)
- Service name typo in connection string
- Target container hasn't started yet (use `depends_on`)

Fix:

```
# Restart all containers (resets networking)
docker compose down
docker compose up -d
```

5. SSL / HTTPS Issues

5.1 SSL Certificate Not Issued (HTTP Works, HTTPS Doesn't)

Symptoms:

- `http://your-domain.com` works
- `https://your-domain.com` shows browser security warning
- Caddy logs show certificate errors

Diagnosis:

```
docker compose logs gateway | grep -i "error\|certificate\|tls\|acme"
```

Fix — Check each step:

```
# Step 1: Is DNS pointing to your VPS?
dig YOUR_SUBDOMAIN.lazuar.com
```

```
# ANSWER section should show your Droplet IP

# Step 2: Is port 80 accessible? (Let's Encrypt needs this!)
# From another machine:
curl -I http://YOUR_SUBDOMAIN.lazuar.com
# Should return something (even a redirect)

# Step 3: Is Cloudflare proxy DISABLED?
# Go to Cloudflare → DNS → your record
# Set proxy status to "DNS Only" (grey cloud, NOT orange cloud)
# ⚠ This is the #1 cause of SSL issues!

# Step 4: Verify Caddyfile domain matches exactly
docker exec kanban-gateway cat /etc/caddy/Caddyfile
# Domain must match DNS record exactly

# Step 5: Restart Caddy to retry certificate
docker compose restart gateway
docker compose logs -f gateway
# Wait 30-60 seconds, look for "certificate obtained successfully"
```

Most common fix: Disable Cloudflare Proxy

Cloudflare DNS Settings:

Type	Name	Content	Proxy Status
A	kanban	167.71.123.45	DNS Only (Grey Cloud) ← MUST be grey!

⚠ **Why?** When Cloudflare proxy (orange cloud) is ON, Caddy can't verify domain ownership with Let's Encrypt because Cloudflare intercepts the challenge. Once Caddy has the certificate, you CAN turn the proxy back on if desired.

5.2 "too many certificates already issued for exact set of domains"

Cause: Let's Encrypt rate limit (5 certificates per domain per week).

Fix:

```
# Wait (rate limit resets weekly)
# OR use Let's Encrypt staging while testing:

# In Caddyfile, add:
{
    acme_ca https://acme-staging-v02.api.letsencrypt.org/directory
}

your-domain.com {
```

```
# ... your config  
}
```

Remove the `acme_ca` line once you're ready for production.

5.3 "ERR_SSL_VERSION_OR_CIPHER_MISMATCH" in Browser

Cause: Usually means Caddy hasn't obtained the certificate yet, or Cloudflare SSL settings conflict.

Fix:

```
# Check if Caddy has certificates  
docker exec kanban-gateway ls /data/caddy/certificates/  
  
# If empty, force renewal:  
docker compose down  
docker volume rm kanban-prod_caddy_data kanban-prod_caddy_config  
2>/dev/null  
docker compose up -d  
  
# Wait 1-2 minutes and check:  
docker compose logs gateway | grep "certificate"
```

If using Cloudflare with proxy enabled:

- Set Cloudflare SSL/TLS mode to **"Full (strict)"**
 - Or disable proxy (grey cloud) and let Caddy handle SSL directly
-

5.4 Certificate Works But Expires / Doesn't Auto-Renew

Cause: Caddy auto-renews, but port 80 might be blocked.

```
# Verify port 80 is open (needed for ACME challenges)  
ufw status | grep 80  
  
# If not:  
ufw allow 80/tcp  
ufw reload  
  
# Check Caddy can reach Let's Encrypt  
docker exec kanban-gateway wget -q --spider https://acme-  
v02.api.letsencrypt.org/directory && echo "OK" || echo "BLOCKED"
```

6. Database Issues

6.1 "Connection refused" to Database

Full error (in API logs):

```
Npgsql.NpgsqlException: Failed to connect to 127.0.0.1:5432
Connection refused
```

For Managed Database:

```
# 1. Check the host in your .env – NOT localhost!
cat ~/kanban-prod/.env | grep DATABASE_URL
# Should be: Host=db-postgresql-xxx.ondigitalocean.com

# 2. Check trusted sources on DigitalOcean
# Go to: Database → Settings → Trusted Sources
# Your Droplet IP MUST be listed

# 3. Test from VPS
apt install -y postgresql-client
psql "host=YOUR_DB_HOST port=25060 dbname=defaultdb user=doadmin
password=YOUR_PASS sslmode=require"

# 4. Test from inside the API container
docker exec kanban-api sh -c "nc -zv YOUR_DB_HOST 25060"
```

For Containerized PostgreSQL:

```
# 1. Is postgres container running?
docker compose ps postgres

# 2. Check postgres logs
docker compose logs postgres

# 3. Check connection string uses service name
# ✅ Correct: Host=postgres;Port=5432;...
# ❌ Wrong: Host=localhost;Port=5432;...

# 4. Is postgres healthy?
docker inspect kanban-postgres --format '{{.State.Health.Status}}'
```

6.2 "password authentication failed"

Full error:


```
FATAL: password authentication failed for user "doadmin"
```

Fix:

```
# Verify credentials
cat ~/kanban-prod/.env | grep DATABASE

# Common mistakes:
# - Extra spaces in password
# - Special characters not escaped
# - Copy-paste added invisible characters

# Rewrite the .env file manually:
nano ~/kanban-prod/.env
# Carefully retype the DATABASE_URL

# Restart
docker compose down
docker compose up -d
```

6.3 "SSL connection is required"

Full error:

```
Npgsql.PostgresException: SSL connection is required. Please specify SSL
mode.
```

Fix: Add SSL settings to connection string in `.env`:

```
DATABASE_URL=Host=...;Port=25060;Database=defaultdb;Username=doadmin;Passwo
rd=...;SSL Mode=Require;Trust Server Certificate=true
```

6.4 Database Migration Fails

Full error (in API logs):

```
An error occurred while migrating the database.
```

Fix:

```
# Check full error
docker compose logs api | grep -A 5 "migration\|error"

# Common causes:
# - Connection string wrong (see 6.1, 6.2)
# - Database already has conflicting schema

# Reset database (DELETES ALL DATA):
# For managed DB:
psql "your_connection_string" -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"

# Restart API to re-run migrations
docker compose restart api
docker compose logs -f api
```

7. GitHub / Git Issues

7.1 "Repository not found" on Clone/Push

Full error:

```
ERROR: Repository not found.
fatal: Could not read from remote repository.
```

Fix:

```
# 1. Check if you have access
# Visit the repo URL in browser: https://github.com/proxeon/bootcamp

# 2. Accept org invitation:
# https://github.com/orgs/proxeon/invitation

# 3. Verify remote URL is correct
git remote -v
# Should show YOUR fork, not the org repo

# 4. If you need to fix the URL:
git remote set-url origin git@github.com:YOUR_USERNAME/bootcamp.git
```

7.2 "Updates were rejected because the tip of your current branch is behind"

Full error:

```
! [rejected]          main -> main (non-fast-forward)
error: failed to push some refs
hint: Updates were rejected because the tip of your current branch is
behind
```

Fix:

```
# Pull first, then push
git pull origin main

# If merge conflicts appear:
# 1. Open conflicting files
# 2. Resolve conflicts (remove <<<<<< markers)
# 3. Stage and commit:
git add .
git commit -m "merge: resolve conflicts"
git push origin main
```

7.3 "fatal: refusing to merge unrelated histories"

Fix:

```
git pull origin main --allow-unrelated-histories
# Resolve any conflicts, then:
git add .
git commit -m "merge: initial sync"
git push origin main
```

8. GitHub Actions / CI-CD Issues

8.1 "403 Forbidden" When Pushing to GHCR

Full error in Actions log:

```
Error: buildx failed: denied: permission_denied: write_package
```

Fix checklist:

1. Organization workflow permissions:

- Go to: <https://github.com/organizations/proxeon/settings/actions>
- Set to: **Read and write permissions**

2. Repository workflow permissions:

- Go to: <https://github.com/proxeon/bootcamp/settings/actions>
- Set to: **Read and write permissions**

3. Package access:

- Go to: <https://github.com/orgs/proxeon/packages>
- Click on the package → Settings → Manage Actions access
- Add your repository with **Write** role

4. Secrets configured:

- Go to: Repository → Settings → Secrets → Actions
- Verify **CR_USER** and **CR_PAT** exist and are correct

8.2 GitHub Actions Not Triggering

Symptoms: You pushed to **main** but no workflow ran.

Check:

```
# 1. Is the workflow file in the right place?
ls .github/workflows/deploy.yml

# 2. Check paths filter – maybe your changes didn't match
# In deploy.yml, the 'paths' filter only triggers on specific folders

# 3. Manually trigger:
# Go to: Repository → Actions → Select workflow → "Run workflow"

# 4. Check if Actions is enabled:
# Repository → Settings → Actions → General → "Allow all actions"
```

8.3 Build Fails in GitHub Actions

Common errors and fixes:

Error	Fix
COPY failed: file not found	Wrong context or file path in workflow
npm ERR! network timeout	Retry (transient) or add retry in workflow
OOMKilled during build	Image too large for runner; optimize Dockerfile
Invalid workflow file	YAML syntax error; validate at yamlvalidator.com

9. DNS Issues

9.1 Domain Not Resolving

Symptom: Browser shows "This site can't be reached" / "DNS_PROBE_FINISHED_NXDOMAIN"

Diagnosis:

```
# Check DNS from your machine
dig YOUR_SUBDOMAIN.lazuar.com

# Check from a public DNS
dig @8.8.8.8 YOUR_SUBDOMAIN.lazuar.com

# Should show your Droplet IP in ANSWER section
# If no ANSWER → DNS record not set or not propagated
```

Fix:

1. Go to Cloudflare DNS dashboard
 2. Verify the A record exists:
 - Type: **A**
 - Name: **YOUR_SUBDOMAIN**
 - Content: **YOUR_DROPLET_IP**
 - Proxy: **DNS Only (grey cloud)**
 3. If just created, wait 1-5 minutes for propagation
-

9.2 DNS Propagation Delay

Symptom: Record is correct in Cloudflare but **dig** shows old/no data.

Fix:

```
# Check if Cloudflare sees it (bypassing cache)
dig @1.1.1.1 YOUR_SUBDOMAIN.lazuar.com

# Force flush (on your machine):
# macOS:
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder

# WSL/Linux:
sudo resolvectl flush-caches

# Wait 5 minutes, Cloudflare propagation is usually fast (<1 min)
```

9.3 Domain Resolves But Site Not Loading

Diagnosis:

```
# 1. DNS resolves correctly?
dig YOUR_SUBDOMAIN.lazuar.com
# ✅ Shows your VPS IP

# 2. VPS is accepting connections?
curl -I http://YOUR_DROPLET_IP
# If timeout → firewall or Docker not running

# 3. Caddy is running?
ssh vps "docker compose -f ~/kanban-prod/docker-compose.yml ps gateway"

# 4. Caddy can serve on port 80?
ssh vps "curl -s -o /dev/null -w '%{http_code}' http://localhost:80"
# Should return 200 or 308 (redirect to HTTPS)
```

10. WSL-Specific Issues

10.1 "WSL is not installed" / "wsl: command not found"

Fix (PowerShell as Admin):

```
wsl --install
# Restart computer
```

If `wsl --install` fails:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-
Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all
/norestart
# Restart computer, then:
wsl --install -d Ubuntu
```

10.2 "WslRegisterDistribution failed with error: 0x80370102"

Cause: Virtualization not enabled in BIOS.

Fix:

1. Restart computer
2. Enter BIOS (press `F2`, `F12`, `Del`, or `Esc` during boot — varies by manufacturer)
3. Find `Intel VT-x`, `Intel Virtualization Technology`, `AMD-V`, or `SVM Mode`
4. Enable it
5. Save and restart

10.3 "The WSL 2 kernel file is not found"

Fix:

```
# Download and install WSL2 kernel update
wsl --update

# If that fails, download manually:
# https://aka.ms/wsl2kernel
```

10.4 Docker Extremely Slow in WSL

Cause: Files on Windows filesystem (`/mnt/c/...`) are very slow from WSL.

Fix:

```
# Always work in the Linux filesystem:
cd ~
# ✓ /home/username/bootcamp (fast – native Linux FS)
# ✗ /mnt/c/Users/name/bootcamp (slow – Windows FS accessed via 9P)

# Clone repos into Linux filesystem:
cd ~
git clone git@github.com:YOUR_USERNAME/bootcamp.git
```

10.5 "Cannot access files" / Permission Issues Between Windows and WSL

```
# If file permissions are wrong:
cd ~/bootcamp
chmod -R 755 .
chmod 600 ~/.ssh/id_ed25519

# If git shows all files as modified (line ending issue):
git config --global core.autocrlf input
git rm --cached -r .
git reset --hard HEAD
```

10.6 WSL Takes Too Much Memory

Fix: Create/edit `%UserProfile%\wslconfig` in Windows:

```
[wsl2]  
memory=4GB  
processors=2  
swap=2GB
```

Then restart WSL:

```
wsl --shutdown
```

11. Resource / Performance Issues

11.1 "OOMKilled" / Server Running Out of Memory

Diagnosis:

```
# Check memory  
free -h  
  
# Check which container was killed  
docker inspect kanban-api --format '{{.State.OOMKilled}}'  
  
# Check container memory usage  
docker stats --no-stream
```

Fix:

```
# Option 1: Reduce memory usage  
# Stop unnecessary services  
docker compose stop watchtower # temporarily  
  
# Option 2: Add swap space (1GB)  
fallocate -l 1G /swapfile  
chmod 600 /swapfile  
mkswap /swapfile  
swapon /swapfile  
echo '/swapfile none swap sw 0 0' >> /etc/fstab  
  
# Verify swap  
free -h  
# Should show Swap: 1.0G  
  
# Option 3: Upgrade Droplet  
# DigitalOcean dashboard → Droplet → Resize → Select larger plan
```

11.2 Disk Space Running Out

```
# Check usage
df -h /
du -sh /var/lib/docker

# Clean up
docker system prune -af
docker builder prune -af
docker volume prune -f

# Check for large log files
find /var -name "*.log" -size +50M -exec ls -lh {} \;

# Truncate docker container logs
truncate -s 0 /var/lib/docker/containers/*/*-json.log
```

11.3 Containers Are Very Slow

```
# Check resource usage
docker stats

# Check if swap is being used heavily (means RAM is full)
free -h

# Check CPU usage
htop

# Check if disk I/O is the bottleneck
iostat -x 1 5 # (install: apt install sysstat)
```

12. Nuclear Options (Start Over)

12.1 Reset Docker Completely (VPS)

```
cd ~/kanban-prod

# Stop everything
docker compose down -v

# Remove ALL Docker resources
docker system prune -af --volumes
docker builder prune -af

# Verify clean
docker ps -a # Should be empty
```

```
docker images      # Should be empty
docker volume ls   # Should be empty

# Re-login and restart
echo $CR_PAT | docker login ghcr.io -u $GH_USERNAME --password-stdin
docker compose up -d
```

12.2 Reset Project Directory (VPS)

```
# Delete and recreate
rm -rf ~/kanban-prod
mkdir -p ~/kanban-prod
cd ~/kanban-prod

# Recreate .env, Caddyfile, docker-compose.yml
# (refer to 006-vps-setup-guide.md sections 6.3-6.5)

# Redeploy
docker compose up -d
```

12.3 Destroy and Recreate Droplet

If nothing else works:

1. **DigitalOcean Dashboard** → Droplets → Your Droplet → **Destroy**
 2. Create a new Droplet (follow [006-vps-setup-guide.md](#))
 3. **Tell the instructor** your new IP (for DNS update)
 4. Re-deploy from scratch
-

12.4 Reset Git Repository Locally

```
# Delete local clone
cd ~
rm -rf bootcamp

# Delete your fork on GitHub:
# https://github.com/YOUR_USERNAME/bootcamp/settings → Delete this repository

# Re-fork from: https://github.com/proxeon/bootcamp
# Re-clone:
git clone git@github.com:YOUR_USERNAME/bootcamp.git
cd bootcamp
git remote add upstream git@github.com:proxeon/bootcamp.git
```

Quick Diagnostic Script

Run this on your VPS to get a complete status report:

```
cat > ~/diagnose.sh << 'SCRIPT'
#!/bin/bash
echo ""
echo "
echo "
echo "
echo "
echo "
echo ""

echo "— 1. SYSTEM —————"
printf " OS:          %s\n" "$(lsb_release -ds 2>/dev/null || cat /etc/os-
release | grep PRETTY | cut -d= -f2)"
printf " Uptime:      %s\n" "$(uptime -p)"
printf " Memory:      %s / %s (Swap: %s)\n" "$(free -h | awk '/Mem:/{print
$3}')" "$(free -h | awk '/Mem:/{print $2}')" "$(free -h | awk
'/Swap:/{print $3}')"
printf " Disk:        %s used of %s (%s)\n" "$(df -h / | awk 'NR==2{print
$3}')" "$(df -h / | awk 'NR==2{print $2}')" "$(df -h / | awk 'NR==2{print
$5}')"
echo ""

echo "— 2. DOCKER —————"
if command -v docker &>/dev/null && docker info &>/dev/null; then
    printf " Status:      ✅ Running\n"
    printf " Version:      %s\n" "$(docker --version | cut -d' ' -f3 | tr -d
',,')"
    printf " Compose:      %s\n" "$(docker compose version | cut -d' ' -f4)"
else
    printf " Status:      ❌ NOT RUNNING\n"
fi
echo ""

echo "— 3. CONTAINERS —————"
if [ -f ~/kanban-prod/docker-compose.yml ]; then
    cd ~/kanban-prod
    docker compose ps --format " {{.Name}}\t{{.Status}}" 2>/dev/null |
column -t -s $'\t'
    echo ""

    # Check for restart loops
    RESTARTING=$(docker compose ps 2>/dev/null | grep -c "Restarting")
    if [ "$RESTARTING" -gt 0 ]; then
        printf " ⚠️ %s container(s) in restart loop!\n" "$RESTARTING"
    fi
else
    echo " ⚠️ ~/kanban-prod/docker-compose.yml not found"
fi
echo ""
```

```

echo "— 4. FIREWALL —————"
if ufw status | grep -q "active"; then
    printf "  Status:    ✅ Active\n"
    printf "  SSH (22):  %s\n" "$(ufw status | grep -q '22/tcp.*ALLOW' &&
echo '✅ Allowed' || echo '❌ BLOCKED')"
    printf "  HTTP (80): %s\n" "$(ufw status | grep -q '80/tcp.*ALLOW' &&
echo '✅ Allowed' || echo '❌ BLOCKED')"
    printf "  HTTPS(443):%s\n" "$(ufw status | grep -q '443/tcp.*ALLOW' &&
echo '✅ Allowed' || echo '❌ BLOCKED')"
else
    printf "  Status:    ⚠️ Inactive (all ports open!)\n"
fi
echo ""

echo "— 5. NETWORK —————"
printf "  Public IP: %s\n" "$(curl -s ifconfig.me)"
printf "  Port 80:   %s\n" "$(ss -tlnp | grep -q ':80 ' && echo '✅
Listening' || echo '❌ Not listening')"
printf "  Port 443:  %s\n" "$(ss -tlnp | grep -q ':443 ' && echo '✅
Listening' || echo '❌ Not listening')"
echo ""

echo "— 6. GHCR LOGIN —————"
if grep -q "ghcr.io" ~/.docker/config.json 2>/dev/null; then
    printf "  Status:    ✅ Logged in\n"
else
    printf "  Status:    ❌ NOT logged in (run: docker login ghcr.io)\n"
fi
echo ""

echo "— 7. HEALTH CHECK —————"
if command -v curl &>/dev/null; then
    API_STATUS=$(curl -s -o /dev/null -w "%{http_code}"
http://localhost/api/health 2>/dev/null)
    printf "  API:       %s\n" "$([ "$API_STATUS" = "200" ] && echo '✅
Healthy' || echo "❌ HTTP $API_STATUS")"

    WEB_STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost/
2>/dev/null)
    printf "  Web:       %s\n" "$([ "$WEB_STATUS" = "200" ] && echo '✅
Healthy' || echo "❌ HTTP $WEB_STATUS")"
fi
echo ""

echo "└──────────────────────────────────┘"
echo ""
SCRIPT

chmod +x ~/diagnose.sh

```

Run anytime with:

```
~/diagnose.sh
```

Asking for Help

When asking the instructor for help, provide:

1. **What you're trying to do** (e.g., "deploy to VPS")
2. **The exact error message** (full text, not paraphrased)
3. **What you already tried**
4. **Output of the diagnostic script** (`~/diagnose.sh`)

Screenshot the error or copy-paste it — don't describe it from memory!